

Evaluating Clustering Algorithms and LLM Prompting Strategies for Architecture Recovery in Apache Lucene Codecs

Christopher Seibel Alexander Makejkin Louis Husemann Mikhail Kiselev
Mohammed Danish Shaikh

June 10, 2026

Contents

1	Introduction	3
2	Study Design	4
2.1	Research Questions	4
2.2	Methodology	4
2.2.1	Dependency Extraction and Preprocessing	4
2.2.2	Semantic-Structural ARC Clustering	4
2.2.3	Hierarchical Summarization	4
2.2.4	LLM Prompting Strategy	5
2.3	Data Analysis Process	5
2.3.1	Structural Normalization and Metadata Extraction	5
2.3.2	Quantitative Profiling Metrics	5
2.3.3	Comparative Similarity Evaluation (RQ1)	6
2.3.4	Qualitative LLM Assessment (RQ2)	6
3	Results	7
3.1	Algorithmic Profiling and Cluster Distribution (RQ1)	7
3.1.1	Structural Similarity Evaluation	8
3.1.2	Semantic-Structural ARC Clustering Evaluation	8
3.2	LLM Prompt Results (RQ2)	10
3.2.1	Zero-Shot	10
3.2.2	One-Shot	11
3.2.3	Zero-Shot Chain-of-Thought	11
4	Discussion	13
4.1	Clustering Performance and Algorithm Variance (RQ1)	13
4.2	Interpretation of Clustering Profiles (RQ1)	13
4.2.1	Analysis of Structural Similarity and Overlap	13
4.3	Semantic vs. Structural Divergence	14
4.4	Impact of Prompting Techniques on LLM-Based Architectural Recovery	14
5	Threats to Validity	16
6	Conclusion and Future Work	17
7	Hours spent per student	18
A	Commands and Technical Reference	19
A.1	Cluster Generation	19
A.2	Metrics Evaluation	19
A.3	Prompts	19
A.3.1	Zero-shot Prompts	19
A.3.2	One-shot Prompts	20
A.3.3	Zero-shot Chain-of-Thought Prompts	22
B	Parts under focus	24

1 Introduction

As software systems evolve, their architectural documentation frequently deviates from the actual implementation. To maintain system integrity and facilitate comprehension, automated architecture recovery techniques are essential. This study evaluates the efficacy of structural clustering algorithms and Large Language Models (LLMs) in extracting and describing the architectural components of an existing software system, specifically focusing on the **Codecs** component of the **Apache Lucene** project.

To address the first research question (**RQ1**), the **ARCADE** tool suite is utilized to extract system-wide dependencies, isolate component-relevant relationships and create clusters describing the architecture. Four distinct structural clustering algorithms are applied to the filtered dependency graph:

- **Weighted Combined Algorithm (WCA)**: A hierarchical clustering approach utilizing edge weights to determine cluster merges.
- **LIMBO**: An information-theoretic algorithm designed to minimize the loss of mutual information during clustering.
- **ACDC**: A heuristic-based algorithm that identifies architectural patterns based on structural connectivity.
- **ARC (Semantic-Structural Clustering)**: A hybrid approach operating at the file level. It fuses a normalized structural dependency graph with a semantic similarity matrix, which is computed via high-dimensional vector embeddings generated by an embedding model (Qodo/Qodo-Embed-1-7B).

The convergence of these algorithms is quantified using Architecture-to-Architecture (**a2a**) and Cluster Coverage (**cvg**) metrics based on the recovered architecture. Technical implementation details and CLI commands are provided in Appendix A.

To address the second research question (**RQ2**), this study investigates generative AI by evaluating how different prompting techniques influence an LLM’s ability to synthesize high-level architectural descriptions from source code. The techniques will be used in the form of hierarchical summarization, enabling us to provide many source code files to an LLM despite context window limitations. This is applied to every cluster created by the clustering algorithms from **RQ1**.

Ultimately, this work aims to quantify the variance in accuracy across different clustering algorithms and determine the impact of prompting strategies on the semantic integrity of AI-generated architectural summaries.

2 Study Design

2.1 Research Questions

- **RQ1:** How do different clustering algorithms vary in their ability to accurately determine the architectural components of a system?
- **RQ2:** How do different prompting techniques impact LLM’s ability to accurately describe architectural components based on source code?

2.2 Methodology

The experimental setup follows a phased workflow designed to recover the architecture of the Apache Lucene Codecs component through a combination of structural clustering and Large Language Model (LLM) analysis.

2.2.1 Dependency Extraction and Preprocessing

The initial phase involves compiling the Java source code to generate a compiled *.jar file.¹ With this the system-wide dependencies are extracted using the ARCADE JavaParser tool, which produces a master .rsf file representing all project relationships. To isolate the scope, the dependencies are filtered such that only relationships where the source or target packages are part under our focus. The parts under our focus can be found in Appendix B.

2.2.2 Semantic-Structural ARC Clustering

To fuse structural dependencies with semantic source code relationships, the Agglomerative Recovery of Components (ARC) methodology is applied. This involves a multi-step feature extraction and matrix fusion process:

- **Semantic Embedding Generation:** For each valid source code file within the target component, semantic vectors are extracted using the Qodo/Qodo-Embed-1-7B model. To ensure hardware efficiency, the model is executed using 4-bit quantization.
- **Similarity Matrices:** A semantic similarity matrix is constructed by computing the cosine similarity between the generated file embeddings. In parallel, a normalized structural similarity matrix is derived from the filtered .rsf dependency graph, calculating the relative package dependency overlap between file pairs.
- **Matrix Fusion and Clustering:** Both matrices are linearly combined using a weighting parameter α , where α dictates the proportion of structural versus semantic data. The resulting similarity matrix is inverted into a distance matrix and processed via *Agglomerative Clustering* (utilizing both *average* and *complete* linkage strategies) to generate the final file-level architecture components.

2.2.3 Hierarchical Summarization

Hierarchical summarization is applied to create a descriptive summary for each cluster. This is necessary because providing all source code files of a cluster to an LLM at once exceeds the context window capacity. For this purpose, every file within a cluster is summarized using an LLM prompt to extract a semantic summary containing key functionalities, core logic, inputs/outputs, and dependencies. These semantic summaries are then combined using another LLM prompt to produce a high-level descriptive summary and a title explaining a module’s overall behavior,

¹Since the current version of Apache Lucene’s Codecs is incompatible with the JavaParser in use, the compiled JAR file and all project investigations were conducted using release version 9.0.0.

architecture, and component interactions within the cluster. To further reduce context window usage, files in subdirectories are also summarized into descriptive summaries before being combined with higher-level files. This approach using two or more LLM prompts for a single cluster can then be compared using different prompt techniques.

2.2.4 LLM Prompting Strategy

The hierarchical summarization will be realized with following prompting techniques:

- **Zero-shot prompting:** Zero-shot prompting provides the task description without examples, relying solely on the model’s pre-trained knowledge. The prompts used can be found in Appendix A.3.1
- **One-shot prompting:** One-shot prompting includes a single input-output example before the actual query to guide the model’s response. The prompts used can be found in Appendix A.3.2
- **Zero-shot Chain-of-Thought Prompting:** Zero-shot Chain-of-Thought prompting instructs the model to generate step-by-step reasoning before producing the final answer, without providing examples. The prompts used can be found in Appendix A.3.3

2.3 Data Analysis Process

The data analysis pipeline is structured into three primary phases: structural normalization, quantitative statistical profiling, and comparative similarity evaluation.

2.3.1 Structural Normalization and Metadata Extraction

Raw architectural outputs from the ARCADE toolset are generated in the *.rsf* format, which represents dependencies line-by-line as `depends Source_Package Target_Package`. To facilitate objective comparison, a post-processing utility is utilized to transform these raw files into structured plain-text artifacts. This utility performs the following operations:

- **Sorting:** Clusters are ordered by cardinality (size), and internal elements are sorted alphabetically to ensure deterministic comparison.
- **Metadata Generation:** For each algorithmic run, the utility extracts a statistical profile used to determine the optimality of the clustering parameters.

Resolution of Granularity Discrepancies A critical methodological challenge arose from the varying input granularity requirements of the applied clustering approaches. The heuristic baselines established (WCA, LIMBO, ACDC) operated on class-level dependencies, inherently preserving inner-class relationships. Conversely, the ARC clustering and the underlying semantic embedding models (RQ2) process source code strictly at the file level (*.java*). To establish mathematically valid comparability for the *a2a* and *cvg* metrics, a deterministic reduction was applied. All inner-class definitions within the original `filtered.rsf` were merged into their respective parent file structures, producing a flattened `filtered-without-innerclasses.rsf`. The structural baselines (WCA, LIMBO, ACDC) were subsequently re-executed on this normalized dependency graph to ensure isomorphic entity sets across all compared architectures.

2.3.2 Quantitative Profiling Metrics

Algorithm selection and parameter tuning are based on a set of distribution metrics that characterize the resulting architecture. These metrics include:

- **Cluster Cardinality:** Total number of identified clusters and the count of singletons (size = 1) versus multi-element clusters (size > 1).
- **Central Tendency:** Average size per cluster across the entire system and specifically for clusters with size > 1.
- **Distribution Variance:** Median size for multi-element clusters and the absolute size of the largest identified cluster.

2.3.3 Comparative Similarity Evaluation (RQ1)

To quantify the variance between clustering algorithms, the processed *.rsf* files are analyzed using the **Architecture-to-Architecture** and **Cluster Coverage** metrics.

- **a2a Similarity:** Measures the global structural distance between two architectures, providing a percentage-based similarity score.
- **Cluster Coverage (cvg):** An asymmetric metric measuring the extent to which clusters in one architecture overlap with clusters in another.

A high percentage in these metrics indicates a high degree of structural convergence between different algorithmic approaches.

2.3.4 Qualitative LLM Assessment (RQ2)

The hierarchical summarization will be applied for all prompting techniques outlined in Section 2.2.4 and will be done with the cluster set produced by the WCA Algorithm using UEM serving as the consistent input across all experiments. The cluster set generated by the WCA Algorithm was specifically selected as it contains a diverse range of cluster sizes, from large clusters spanning multiple files to small clusters consisting of a single file, making it well-suited for evaluating the effect of prompting techniques across varying levels of complexity. The evaluation is intentionally limited to a single cluster set, as the results yielded by different cluster sets under the same prompting technique are expected to be structurally similar. Consequently, the choice of cluster does not significantly influence the comparative outcomes, making a multi-cluster analysis redundant for the purposes of this research question.

The prompts will be generated on the university’s High Performance Computing (HPC) clusters with the **ibm-granite/granite-34b-code-instruct-8k** Model by IBM Research. The model was chosen because of its specialised training on programming languages and source code, making it well-suited for code-oriented prompt generation. Additionally, its 34B parameter size fits within the memory constraints of the available 2×A100 GPU hardware. Finally, since the model weights are publicly released and versioned, all experiments remain fully reproducible.

The script used for the assessment can be found in this repository²

²https://github.com/lhusemann/ds_for_se_a1/tree/main/Week_4/hcp-scripts

3 Results

3.1 Algorithmic Profiling and Cluster Distribution (RQ1)

The statistical profiles of the generated architectures for WCA and LIMBO are summarized in Table 1 and 2.

Table 1: Quantitative Profiling of Clustering Algorithms (Targets: 70 and 50 Clusters)

	Target: 70 Clusters			Target: 50 Clusters		
	WCA (UEMNM)	WCA (UEM)	LIMBO	WCA (UEMNM)	WCA (UEM)	LIMBO
Cluster Cardinality						
Total number of clusters	70	70	70	50	50	50
Number of clusters (size = 1)	48	45	26	41	41	12
Number of clusters (size > 1)	22	25	44	9	9	38
Central Tendency						
Avg. size per cluster	2.70	2.70	2.70	3.78	3.78	3.78
Avg. size (clusters > 1)	6.41	5.76	3.70	16.44	16.44	4.66
Distribution Variance						
Median size (clusters > 1)	6	6	2	16	13	4
Largest cluster size	11	10	41	28	42	41

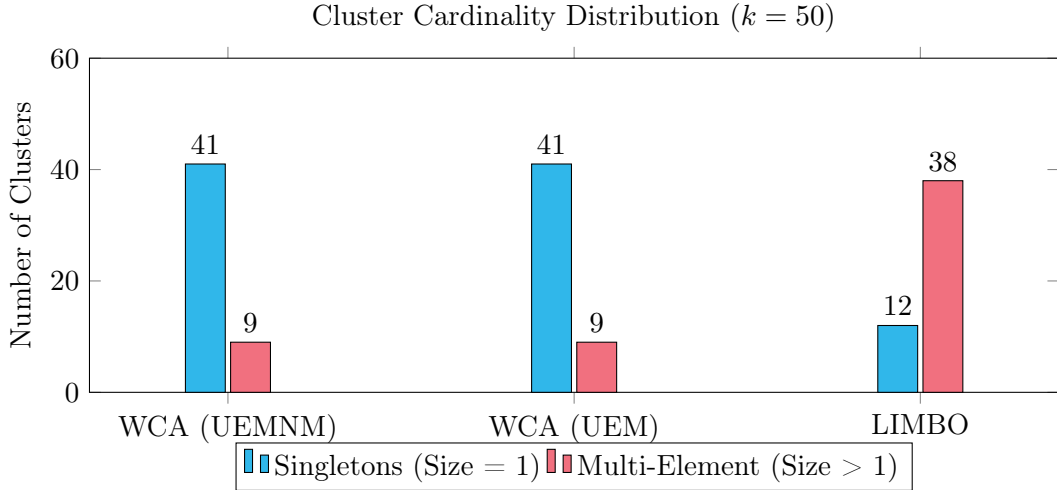


Figure 1: Comparison of cluster composition at a target granularity of $k = 50$. WCA primarily isolates singletons, whereas LIMBO achieves a broader distribution of multi-element clusters.

Table 2: Quantitative Profiling of the Clustering Algorithm ACDC

ACDC	
Cluster Cardinality	
Total number of clusters	5
Number of clusters (size = 1)	0
Number of clusters (size > 1)	5
Central Tendency	
Avg. size per cluster	37.8
Avg. size (clusters > 1)	37.8
Distribution Variance	
Median size (clusters > 1)	26
Largest cluster size	97

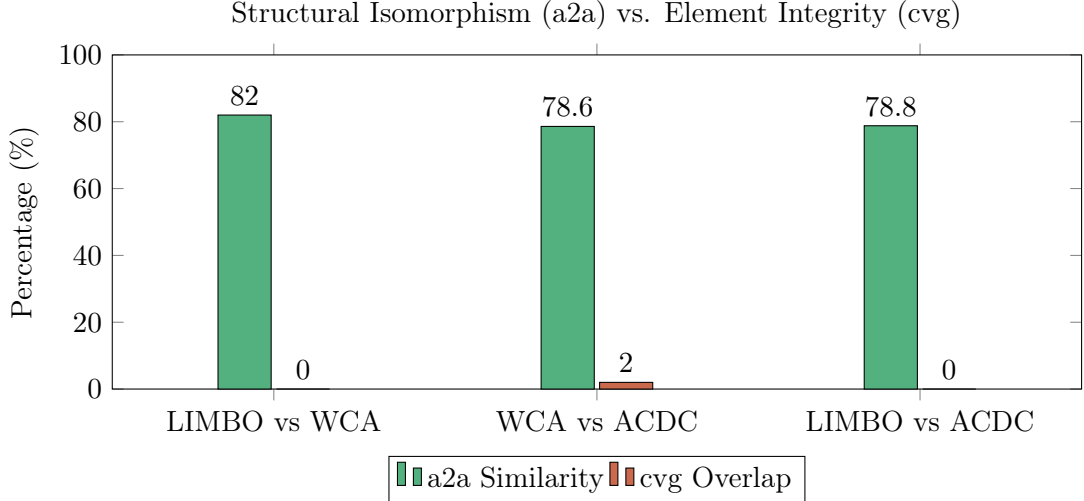


Figure 2: Visualization of the metric discrepancy: Despite high topological congruence ($a2a \approx 80\%$), the algorithms fail to converge on the concrete allocation of elements ($cvg \approx 0\%$).

3.1.1 Structural Similarity Evaluation

Table 3 illustrates the structural similarity and overlap across the algorithm pairs. To provide a comprehensive comparison, the WCA algorithm is explicitly differentiated by its underlying similarity measures: UEM and UEMNM. All evaluations were conducted on architectures partitioned into exactly 50 clusters.

Table 3: Architecture-to-Architecture (a2a) and Cluster Overlap (cvg)

Cluster Count 50	a2a Similarity	cvg (A $\rightarrow$ B)	cvg (B $\rightarrow$ A)
LIMBO vs WCA (UEM)	82.02%	0.0%	0.0%
LIMBO vs WCA (UEMNM)	82.23%	0.0%	0.0%
WCA (UEM) vs ACDC	78.59%	2.0%	5.26%
WCA (UEMNM) vs ACDC	78.15%	2.0%	5.26%
LIMBO vs ACDC	78.81%	0.0%	0.0%

3.1.2 Semantic-Structural ARC Clustering Evaluation

To further evaluate architecture recovery performance, we applied ARC clustering, which combines both structural dependencies and semantic similarity derived from source code embeddings. ARC operates at file level rather than class level. Therefore, the baseline WCA, LIMBO, and ACDC architectures were regenerated using a filtered dependency graph without inner classes to ensure comparability. Multiple ARC configurations were evaluated by adjusting the weighting parameter α , linkage strategy, and target cluster count. The parameter α controls the contribution of structural versus semantic similarity within the combined similarity matrix.

Structural Similarity Evaluation: $k = 50$ vs. $k = 57$ In this evaluation, the Qodo-embedding-based ARC clustering outputs were tested against established baseline heuristics (ACDC, LIMBO, WCA). Regardless of the specific hyperparameter configuration—including the structural-semantic weight ($\alpha \in [0.25, 0.75]$) and the linkage strategy (e.g., average, complete)—all QODO-based ARC variants exhibit structural isomorphism at a fixed target cluster count (k). Therefore, their results are presented in an aggregated format. Table 4 contrasts the outcomes for cluster counts 50 and 57.

Table 4: Comparison of Similarity Metrics (a2a) and Overlap (cvg)

Cluster Count	Algorithm Pair	a2a Similarity	cvg (A $\rightarrow$ B)	cvg (B $\rightarrow$ A)
$k = 50$	ARC vs LIMBO	78.33%	0.80%	0.80%
	ARC vs WCA (UEM)	70.20%	0.10%	0.10%
	ARC vs WCA (UEMNM)	70.69%	0.10%	0.10%
	ARC vs ACDC	61.33%	0.00%	0.00%
$k = 57$	ARC vs LIMBO	80.71%	0.93%	0.93%
	ARC vs WCA (UEM)	72.14%	0.09%	0.09%
	ARC vs WCA (UEMNM)	72.14%	0.11%	0.11%
	ARC vs ACDC	59.69%	0.00%	0.00%

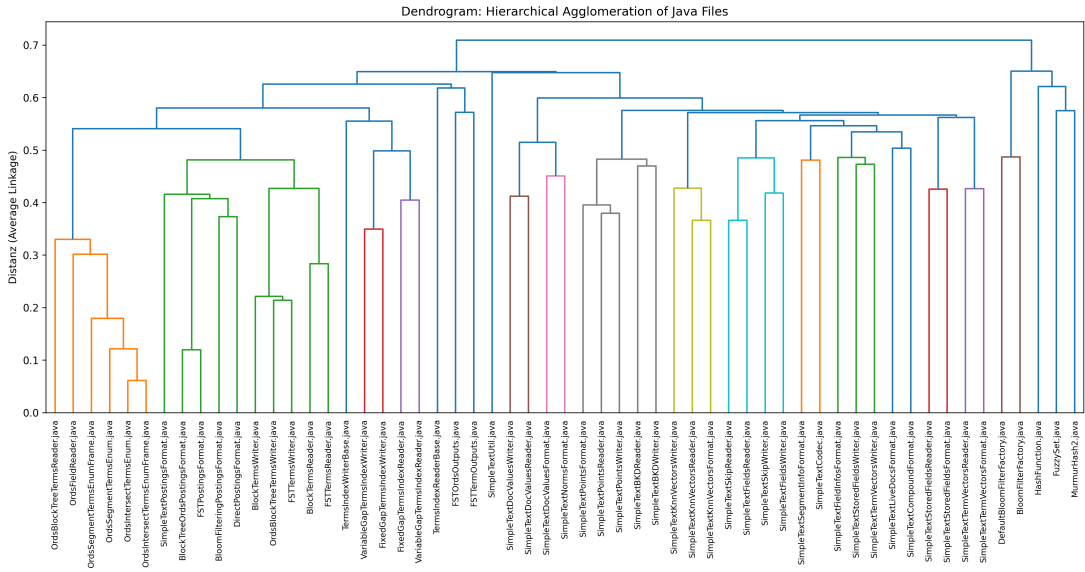


Figure 3: ARC Clustering: $\alpha = 0.50$, Average Linkage

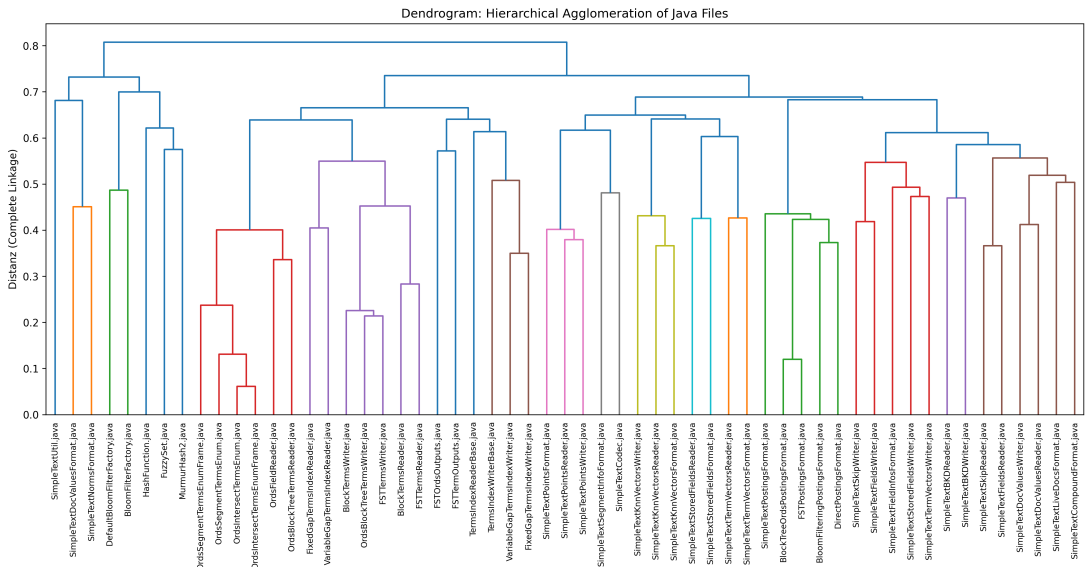


Figure 4: ARC Clustering: $\alpha = 0.50$, Complete Linkage

The comparison reveals significant topological variances: Average Linkage tends to form highly asymmetric, chain-like structures (chaining effect), whereas Complete Linkage enforces more compact and balanced cluster subgraphs.

3.2 LLM Prompt Results (RQ2)

The complete summarization outputs for all evaluated prompting techniques are available in this repository³ with each `.txt` file named according to the ID of the corresponding cluster. The following subsections present the results of each prompting technique individually, highlighting key observations regarding structural consistency and content quality.

3.2.1 Zero-Shot

The zero-shot prompting results exhibited a high degree of inconsistency across all evaluated clusters. The structure of the outputs varied considerably, with some responses omitting the mandated **Title** and **Summary** fields entirely. Certain outputs, such as the one shown for Cluster 1 in Table 5, produced a summary but largely disregarded the content of most files within the cluster. Others, as observed in Cluster 40, generated individual summaries for a subset of the files rather than covering the cluster comprehensively.

Clusters containing a single file generally yielded more coherent results, though significant variation in output length was still observed. For instance, the description for Cluster 22 spanned multiple paragraphs, whereas Cluster 31 received only four brief sentences.

A particularly notable result was observed in Cluster 2, where the model disregarded the prompt entirely and instead rewrote portions of the file as python code, subsequently describing that code rather than the original content.

Overall, zero-shot prompting produces inconsistent and unreliable hierarchical summaries, with the model frequently deviating from the required output structure and in some cases ignoring the prompt altogether.

Table 5: Example Zero-Shot Output (Cluster 1)

Here is a summary of the provided code:

The code is a part of Lucene, a search engine library. It implements a terms index reader for the `VariableGapTermsIndexWriter`. The index reader is used to read terms from the index and provides functionality to seek to a specific term and retrieve the corresponding term’s ord. The index reader also implements `Accountable`, which allows for estimation of memory usage. The code reads the index file, which contains information about terms and their corresponding positions in the index. It then creates a `FieldIndexData` object for each field in the index, which contains the FST (Finite State Transducer) data structure. The FST is used to efficiently navigate through the index and retrieve terms.

The `IndexEnum` class implements the `FieldIndexEnum` interface, which provides methods to iterate through the terms in the index and retrieve their corresponding ord values. The `seek` method seeks to a specific term in the index, while the `next` method retrieves the next term in the index.

The code also provides methods to estimate the memory usage of the index and its child resources.

Overall, this code provides a high-performance and memory-efficient way to read terms from the index and navigate through the index efficiently.

³https://github.com/lhusemann/ds_for_se_a1/tree/main/Week_4/generated_files/wca_UEM

3.2.2 One-Shot

One-shot prompting demonstrated a marked improvement in structural consistency compared to zero-shot prompting. All outputs adhered to the format established by the example prompt, consistently including the mandated **Title** and **Summary** fields. However, despite this structural regularity, the technique exhibited significant shortcomings in terms of content quality.

For clusters containing multiple files, the model focused on a single class from within one file, disregarding the remainder of the cluster entirely, as observed in Table 6 for Cluster 2. Clusters consisting of a single file generally performed more reliably, with outputs such as that of Cluster 9 producing appropriate and relevant descriptions.

A particularly striking finding was the prevalence of outputs that had no relation to their corresponding cluster. In several cases, the model appeared to reproduce content derived from the example prompt rather than the provided cluster, effectively ignoring the actual input. This behaviour was observed in both multi-file clusters such as Cluster 1 and single-file clusters such as Cluster 22.

Overall, while one-shot prompting successfully enforced a consistent output structure, it introduced a critical reliability issue whereby the model defaulted to the example content rather than the target input, severely undermining the quality of the generated summaries.

Table 6: Example One-Shot Output (Cluster 2)

****Title:**** Direct Automaton IntersectTermsEnum — Automaton-based Term Enumerator

****Summary:**** The Direct Automaton IntersectTermsEnum is a custom implementation of the TermsEnum interface that efficiently intersects automata-based acceptors with a list of terms. It is designed to provide fast and memory-efficient term enumeration by directly traversing the automaton states without the need for backtracking.

The implementation uses a DirectIntersectTermsEnum.State class to represent the current state of the automaton traversal, including the current state and the term position. It initializes the state with the initial state of the automaton and the start term. The next() method advances the state by incrementing the term position and checking if the current state is accepting. If not, it steps the automaton with the current term and updates the state accordingly. The termState() method returns the current state as a OrdTermState object. The seekCeil() method is not implemented in this version of the code, but it can be added by using the skip pointers to efficiently navigate through the terms.

Overall, the Direct Automaton IntersectTermsEnum provides a memory-efficient and fast way to intersect automata-based acceptors with a list of terms, making it suitable for large-scale indexing and search applications.

3.2.3 Zero-Shot Chain-of-Thought

Zero-shot chain-of-thought prompting similarly demonstrated a marked improvement in structural consistency compared to zero-shot prompting. All outputs included a summary and title as asked in the prompt. However, in line with one-shot prompting, this technique also exhibited shortcomings in terms of content quality.

Coverage of multi-file clusters was generally more comprehensive than in previous techniques, with most outputs addressing the cluster as a whole rather than focusing on a subset of its contents. Exceptions were nonetheless present, as seen in Cluster 1, where the summary was limited to a single file despite the cluster containing multiple. Clusters consisting of a single file were handled reliably for the most part.

A consistent and notable pattern observed across all outputs was the composition of the generated titles. Rather than producing descriptive or semantically meaningful titles, the model

defaulted to using filenames present within the cluster. Single-file clusters received the corresponding filename as their title, while multi-file clusters were assigned one or two filenames as a title, as observed in Table 7 for Cluster 2, even though the summary itself addressed additional files within the cluster. This suggests that the chain-of-thought reasoning process influenced the content of the summary more effectively than it did the title generation.

Overall, zero-shot chain-of-thought prompting proved more reliable than both zero-shot and one-shot prompting in terms of content coverage, yet it has a tendency to generate filename-based titles rather than meaningful ones.

Table 7: Example Chain-of-Thought Output (Cluster 2)

Title: SimpleTextSkipWriter and SimpleTextSkipReader **Summary:** The SimpleTextSkipWriter class is used to write skip data in a plain text format for a Lucene index. It buffers skip data and writes it to an output stream, optimizing the retrieval of documents in the index. The SimpleTextSkipReader class is a reader for Lucene’s SimpleTextFieldsWriter. It reads in the fields from the input and returns them as a Terms object. The Terms object can be iterated over to retrieve the terms and their associated information, such as doc frequency, total term frequency, and postings. The postings can be iterated over to retrieve the doc IDs, term frequencies, positions, offsets, and payloads. The SimpleTextFieldsReader also handles skip lists, which are used to efficiently navigate through the postings. The skip lists contain information about the postings at certain intervals, allowing for faster navigation to specific terms. The SimpleTextFieldsReader also handles terms enumerations, which allow for efficient navigation through the terms in the index.
--

4 Discussion

This section interprets the quantitative findings from the structural and semantic recovery phases, evaluating the efficacy of the applied clustering algorithms in determining the architectural components of the Apache Lucene Codecs module.

4.1 Clustering Performance and Algorithm Variance (RQ1)

To answer RQ1, we analyze the structural and semantic distance between the architectures generated by WCA, LIMBO, ACDC, and ARC. Based on the pairwise *a2a* (Architecture-to-Architecture) similarity and *cvg* (Coverage) metrics, the algorithms exhibit distinct behavior in terms of module cohesion and granularity:

- **Structural Baseline Consensus:** The highest similarity among purely structural algorithms was observed between LIMBO and WCA with an *a2a* similarity of 82.23%, indicating a shared reliance on static class dependencies.
- **Impact of Semantic Edge Weighting (α):** Integrating semantic similarity via ARC showed that the weighting parameter α shifts cluster boundaries. At $\alpha = 0.50$, the balance between structural dependencies and source-code vocabulary alignment resulted in an *a2a* similarity of 70.20% compared to the WCA (UEM) structural heuristic.
- **Linkage Criteria Effects:** As visualized in the dendrograms (see Fig. 3 and Fig. 4), Average Linkage tended to form more asymmetric sub-clusters, whereas Complete Linkage forced stricter, more compact boundaries, directly altering the structural recovery consensus.

4.2 Interpretation of Clustering Profiles (RQ1)

An analysis of the clustering profiles indicates distinct algorithmic behaviors in architecture recovery. The pattern-based algorithm ACDC (Table 2) acts as a heuristic baseline. By dynamically resolving to 5 sub components without producing any singleton clusters, ACDC demonstrates aggressive orphan adoption and subgraph dominance. This results in highly abstract, large-scale subsystems with a median size of 26 and maximum 97, rather than fine-grained modules.

In contrast WCA and LIMBO exhibit strong sensitivity to the target granularity (k). At a fine-grained target of $k = 70$ (Table 1), WCA (particularly with UEMNM) yields a structurally cohesive distribution. It produces a higher median cluster size (6) and a bounded maximum cluster size (11), successfully isolating moderately sized, functional modules. Conversely, LIMBO demonstrates an unbalanced bottom-up aggregation: it forms a massive monolithic cluster (size 41) containing structurally disparate elements, while fragmenting the remainder into a high volume of micro-clusters (median size 2).

Reducing the target granularity to $k = 50$ (Table 1) exacerbates algorithmic biases and alters the topology significantly. WCA isolates an extreme number of singleton classes (41 out of 50 clusters are of size 1), forcing the remaining system entities into merely 9 dense clusters (maximum size up to 42 for UEM). This indicates that WCA struggles to establish mid-level cohesion at this abstraction level, instead isolating outliers and heavily clumping the core architecture. LIMBO, while maintaining its primary monolithic cluster (which remains static at size 41), provides a more distributed secondary architecture, grouping the remaining entities across 38 multi-element clusters with a more stable median size of 4.

4.2.1 Analysis of Structural Similarity and Overlap

As detailed in Table 3, the Architecture-to-Architecture (*a2a*) evaluation reveals the highest macro-structural similarity (up to 82.23%) between LIMBO and the WCA variants. ACDC

exhibits a marginally lower, yet consistent, similarity score ($\sim 78.5\%$) when compared to both hierarchical approaches.

However, the global structural similarity masks severe discrepancies in specific element allocation, which are starkly exposed by the Cluster Coverage (**cvg**) metric. Across almost all baseline comparisons, the overlap registers at exactly 0.0%. Most notably, the coverage between LIMBO and WCA, as well as LIMBO and ACDC, indicates a complete lack of identical clusters. While the algorithms construct analogous macro-topologies (similar cluster size distributions), they populate these clusters with entirely disjointed entity sets.

The only measurable, albeit marginal, overlap exists between WCA and ACDC (2.0% vs. 5.26%). The inherent asymmetry of this cvg variance is heavily influenced by the differing total cluster counts (fixed $k = 50$ for WCA vs. dynamic $k = 5$ for ACDC). Overall, the near-zero coverage across all structural baselines highlights that file-level dependency analysis (without inner classes) creates highly volatile partitioning landscapes. Heuristic (ACDC), information-theoretic (LIMBO), and edge-weight-based (WCA) algorithms fail to converge on identical micro-architectures, despite operating on the exact same normalized `.rsf` graph.

4.3 Semantic vs. Structural Divergence

The comparison of semantic clustering outputs against structural baselines yields critical insights regarding the model’s architecture recovery performance:

- **Scaling Effects ($k = 50 \rightarrow k = 57$):** For LIMBO and WCA, a higher cluster granularity ($k = 57$) correlates with an increase in *a2a Similarity* by approximately 2 percentage points. This indicates that the LLM-generated partitioning converges more closely with the structural heuristics at a finer granularity. Conversely, the similarity to ACDC decreases slightly, attributable to the unpredictability of ACDC’s dynamic partitioning when evaluated against fixed cluster constraints.
- **Structural Isomorphism vs. Element Integrity:** The most prominent result is the significant discrepancy between high *a2a* ($> 70\%$) and marginal *cvg* ($< 1\%$).
 - The **a2a Similarity** indicates that the topology and relative distribution of the clusters are highly consistent with LIMBO/WCA. The LLM’s system comprehension regarding functional groupings aligns with the baseline algorithms on a macro-architectural level.
 - However, the **cvg (Coverage)** reveals that the exact allocation of individual entities (classes/files) diverges almost completely. While the algorithms construct similar "containers" (clusters), they populate them with entirely different elements.
- **Methodological Cause:** The baseline algorithms (particularly LIMBO and WCA) operate primarily on *Structural Dependencies* (e.g., method calls, inheritance). In contrast, the LLM (QODO) utilizes *Semantic Embeddings* (e.g., identifiers, comments, natural language semantics). The results demonstrate that while semantic coherence and structural coupling generate analogous abstraction levels (cluster count/size distribution) within this dataset, they act orthogonally concerning the concrete allocation of elements.

4.4 Impact of Prompting Techniques on LLM-Based Architectural Recovery

This section discusses the results of the three evaluated prompting techniques and assesses their suitability for the task of hierarchical summarization in the context of architectural recovery.

- **Zero-Shot Prompting:** The results indicate that zero-shot prompting is wholly inadequate for this task. Outputs were highly inconsistent in both structure and content, with

the model largely disregarding the contents of most files within a given cluster. The occurrence of anomalous outputs, such as the generation of a python code in place of a structured summary, further underlines the unreliability of this technique for architectural recovery.

- **One-Shot Prompting:** One-shot prompting similarly proved unsuitable, despite demonstrating an improvement in structural consistency. While the technique successfully enforced a uniform output format, it introduced a critical reliability issue in the form of in-context overfitting, whereby the model defaulted to the content of the example prompt rather than the provided input. Combined with the tendency to disregard portions of the cluster content, a shortcoming shared with zero-shot prompting, this renders one-shot prompting equally unfit for the task.
- **Chain-of-Thought (CoT) Prompting:** Zero-shot chain-of-thought prompting produced the most reliable results of the three techniques. Structural consistency was maintained across all outputs, and in most cases the model addressed the cluster as a whole rather than focusing on a subset of its contents. Nevertheless, the technique is not without its limitations, most notably its tendency to generate filename-based titles rather than semantically meaningful ones, and occasional incomplete coverage of cluster contents.

Ultimately, the findings indicate that simple code-to-text prompting (Zero-Shot, One-Shot) is insufficient for complex architectural recovery. Explicit reasoning steps (CoT) are strictly required to elevate the LLM’s focus from raw syntax translation to semantic architecture synthesis.

5 Threats to Validity

- **Internal Validity:** An identified bug in the ARCADE Clusterer prevents the correct functionality of the `stopthreshold` argument. Consequently, the algorithms default to generating at least 50 clusters, limiting comparability for smaller target cluster counts. Additionally, it was assumed that the model occasionally ignores files within a cluster. However, this could also be a result of the context window limit being reached, causing some files to be omitted from the input. Although the context window should be sufficiently large to handle the input, this was not explicitly tested and therefore cannot be fully ruled out as a source of error.
- **Construct Validity:** The structural similarity metrics Architecture-to-Architecture (**a2a**) and Cluster Coverage (**cvg**) were the only two metrics given. These are not directly suitable for deciding if a clustering is better or worse, as they only provide similarity comparisons between clusterings. To provide conclusive answers to that research question, other suitable metrics should need to be used, such as ones that measure the accuracy of a clustering compared to its source code, to accurately determine the architectural components of the system. Additionally, the **cvg** metrics' th_{cvg} parameter was not customizable using the provided tool, causing the score to be 0% for most cluster comparisons, despite there being noticeable similarity based on comparing the clusters between two clusterings by hand. This leads us to believe that the value is set rather high for the given scope and limitation induced by the tool (min. 50 clusters as mentioned above). As such, a lower value for that parameter could result in more diverse outputs, which would lead to more accurate conclusions.
- **External Validity:** We used Apache Lucene version 9.0.0 because ARCADE JavaParser was incompatible with newer Java versions. This limits how well our findings apply to current Lucene releases, which may have evolved since 9.0.0. Additionally, by focusing only on the Codecs Module, our results may not generalize to other Lucene modules or systems with different architectural structures. Furthermore, as our analysis is based on a Java project, the conclusions may not extend to software written in other programming languages. Finally, only a single model was used for the hierarchical summarisation, which limits the generalisability of the results, as different models may yield varying levels of performance.
- **Dependency Scope and Entity Truncation:** During the extraction of semantic embeddings, an ambiguity in resolving target dependencies from the `.rsf` file was identified. While the filtered dependency graph referenced 98 distinct entities across the global `lucene/lucene/` directory, only 57 files were strictly localized within the assigned `codecs` component focus area. To prevent embedding noise and mitigate cross-directory naming collisions, the semantic analysis scope was rigidly bounded to the `codecs` package. Consequently, out-of-package dependencies—while structurally present in the graph—were semantically truncated. This localized extraction potentially skewed the combined similarity matrix in the ARC clustering setup, artificially inflating the structural weight of missing external semantics.
- **Granularity Normalization Impact:** As detailed in Section 2.3.1, merging inner-class definitions into parent file structures forces an artificial isomorphic mapping. This abstraction removes low-level coupling resolution, which inherently limits the maximum achievable Cluster Coverage (*cvg*) score and represents a threat to construct validity.

6 Conclusion and Future Work

Regarding RQ1: The initial work to answer the research question involved compiling and then extracting structural information from the project of interest (**Apache Lucene - Codecs**) via the **ARCADE** tool. The collected class-based dependencies are then filtered, processed into file-based dependencies, and multiple structural clustering algorithms (**WCA**, **LIMBO**, **ACDC**) are applied to generate clusterings. Additionally, a hybrid algorithm introducing semantic information (**ARC**) is applied at the file level. Finally, two metrics (**a2a**, **cvg**) are used to derive structural similarity between clusterings.

The resulting information extracted from this process shows that the structural clustering algorithms achieve a similar **a2a** score among each other, however the underlying differences in each algorithm’s theoretical baseline show differences in granularity and cluster sizes, leading to low structural overlap and ultimately a low **cvg** score. The inclusion of semantic information via **ARC** offers an alternative approach and results in more cohesive clusters.

The path forward clearly indicates a need for a hybrid approach that takes in as much diverse information as possible, and not constrained to just inter-file dependencies. Including semantic information gained from artifacts such as source code comments, commit messages, discussion threads, etc., will improve the results further and lead to more cohesive and comprehensible clusters, which is ultimately the goal of such a task.

Regarding RQ2: In conclusion, simple code-to-text prompting approaches such as **zero-shot** and **one-shot prompting** prove insufficient for complex architectural recovery tasks, as the model either ignores significant portions of the input or produces hallucinated content such as fabricated code or template-derived outputs. **Zero-shot chain-of-thought prompting** represents the most reliable technique among those tested, though its limitations in title generation and content coverage indicate that further refinement is necessary. Ultimately, none of the evaluated techniques produced consistently acceptable results across all clusters. It is therefore recommended that future work explores more advanced prompting strategies.

One path worth investigating is standard few-shot prompting, to determine whether providing additional examples mitigates the template overfitting observed in one-shot prompting. One-shot or few-shot chain-of-thought prompting also presents a promising direction, as the inclusion of examples may improve title generation without sacrificing the structural reliability demonstrated by zero-shot chain-of-thought prompting.

Furthermore, as this study was conducted using a single model, the findings may not generalize across different large language models. Comparative research across multiple models would therefore provide a more comprehensive understanding of the role model choice plays in architectural recovery tasks. Finally, given the rapid pace of advancement in the field, repeating this evaluation with more capable future models represents a worthwhile extension of this work.

7 Hours spent per student

Name	W1	W2	W3	W4	W5	W6
Christopher Seibel	1	4	8	8	2	0
Alexander Makejkin	4	2	3.5	2	3	3
Louis Husemann	3	3	7	18	8	4
Mikhail Kiselev	3	3	3	2	4	3
Mohammed Danish Shaikh	3	3	4	2	4	3

A Commands and Technical Reference

A.1 Cluster Generation

WCA:

```
java -Xmx4096m -jar arcade_core_clusterer.jar language=java \
deps=filtered.rsfc projname=lucodecs projversion=9.0.0 \
projpath=output-cluster packageprefix="org.apache.lucene.codecs" \
algo=WCA measure=<UEMNM or UEM> \
serial=STEPCount serialthreshold=1
```

LIMBO:

```
java -Xmx4096m -jar arcade_core_clusterer.jar language=java \
deps=filtered.rsfc projname=lucodecs projversion=9.0.0 \
projpath=output-cluster packageprefix="org.apache.lucene.codecs" \
algo=LIMBO measure=IL \
serial=STEPCount serialthreshold=1
```

ACDC:

```
java -jar arcade_core-ACDC.jar filtered.rsfc acdc.rsfc
```

A.2 Metrics Evaluation

a2a & cvg :

```
java -jar arcade_core_A2a.jar ./file1.rsfc ./file2.rsfc
java -jar arcade_core_Cvg.jar ./file1.rsfc ./file2.rsfc
```

A.3 Prompts

A.3.1 Zero-shot Prompts

Prompt for semantic summary:

System Prompt

You are a helpful data scientist for software engineering. Your job is to explain the functionality of the provided code by extracting a semantic summary from the file: (Key functionality, Core logic, Inputs/Outputs, and Dependencies).

User Prompt

Please analyze the following source code:

```
<source_code>
{content of source code file}
</source_code>
```

Prompt for descriptive summary:

System Prompt

You are a helpful data scientist for software engineering. Your job is to create a descriptive summary and a title explaining a software module's overall behaviour, architecture, and how the components interact within the cluster.

User Prompt

Please create a high-level descriptive summary for the following:

```
<semantic_summaries>
{content of available semantic summaries}
</semantic_summaries>

<descriptive_summaries>
{content of available descriptive summaries}
</descriptive_summaries>
```

A.3.2 One-shot Prompts

Prompt for semantic summary:

System Prompt

You are a helpful data scientist for software engineering. Your job is to explain the functionality of the provided code by extracting a semantic summary from the file: (Key functionality, Core logic, Inputs/Outputs, and Dependencies).

User Prompt

Please analyze the following source code:

```
<source_code>
import java.util.List;
import java.util.ArrayList;
public class EvenNumberFilter {
    public List<Integer> filterEvens(List<Integer> numbers) {
        List<Integer> result = new ArrayList<>();
        for (int n : numbers) {
            if (n % 2 == 0) {
                result.add(n);
            }
        }
        return result;
    }
}
\}
</source_code>
```

Assistant Prompt

****Key Functionality:****

Filters a list of integers, returning only the even numbers.

****Core Logic:****

Iterates over each element in the input list and checks divisibility by 2 using the modulo operator. Matching elements are collected into a new list.

****Inputs/Outputs:****

- Input: 'List<Integer> numbers' --- a list of integers to be filtered
- Output: 'List<Integer>' --- a new list containing only the even integers from the input

****Dependencies:****

- 'java.util.List' --- interface for the input and output list types
- 'java.util.ArrayList' --- concrete list implementation used to collect results

User Prompt

Please analyze the following source code:

```
<source_code>
{content of source code file}
</source_code>
```

Prompt for descriptive summary:

System Prompt

You are a helpful data scientist for software engineering. Your job is to create a descriptive summary and a title explaining a software module's overall behaviour, architecture, and how the components interact within the cluster.

User Prompt

Please create a high-level descriptive summary for the following:

```
<semantic_summaries>
```

****Key Functionality:****

Filters a list of integers, returning only the even numbers.

****Core Logic:****

Iterates over each element in the input list and checks divisibility by 2 using the modulo operator. Matching elements are collected into a new list.

****Inputs/Outputs:****

- Input: 'List<Integer> numbers' --- a list of integers to be filtered
- Output: 'List<Integer>' --- a new list containing only the even integers from the input

****Dependencies:****

- 'java.util.List' --- interface for the input and output list types
- 'java.util.ArrayList' --- concrete list implementation used to collect results

```
</semantic_summaries>
```

```
<descriptive_summaries>
```

```
**Title:** predicates --- Parity Predicate Utilities
```

```
**Summary:**
```

The predicates subdirectory provides reusable predicate logic for numeric classification. Its primary component is a ParityChecker class that exposes static boolean methods for determining whether an integer is even or odd. It acts as a pure logic layer with no state or I/O, designed to be consumed by higher-level filtering components. All methods are stateless and side-effect free, depending only on primitive arithmetic.

```
</descriptive_summaries>
```

Assistant Prompt

```
**Title:** EvenNumberFilter --- Integer Parity Filter Utility
```

```
**Summary:**
```

EvenNumberFilter is a single-responsibility filtering module that isolates even integers from an arbitrary input list. It is composed of two layers: a ‘predicates’ subdirectory that houses the reusable parity logic, and the top-level filter class itself which orchestrates the linear scan and result collection.

The architecture follows a clean separation of concerns --- predicate evaluation is delegated entirely to the stateless ‘ParityChecker’ utilities in the ‘predicates’ layer, while EvenNumberFilter owns the iteration and accumulation logic using a standard ArrayList. Data flows in one direction: the input list is passed down to the filter, elements are tested via the predicate layer, and matching values are collected into a new list that is returned to the caller.

There is no shared mutable state anywhere in the module, making it deterministic and safe for concurrent use. Its only external dependencies are the standard Java Collections API, making it a self-contained leaf cluster with minimal coupling to the broader codebase.

User Prompt

Please create a high-level descriptive summary for the following:

```
<semantic_summaries>
```

```
{content of available semantic summaries}
```

```
</semantic_summaries>
```

```
<descriptive_summaries>
```

```
{content of available descriptive summaries}
```

```
</descriptive_summaries>
```

A.3.3 Zero-shot Chain-of-Thought Prompts

Prompt for semantic summary:

System Prompt

You are a helpful data scientist for software engineering. Your job is to explain the functionality of the provided code by extracting a semantic summary.

Before producing your final summary, reason through the code step by step:

1. Read through the code and identify what each class and method does
2. Trace the core logic and any data transformations
3. Identify all inputs, outputs, and return types
4. Note any imported or external dependencies

Then, present your final summary using exactly these four sections:

- Key Functionality
- Core Logic
- Inputs/Outputs
- Dependencies

User Prompt

Please analyze the following source code:

```
<source_code>
{content of source code file}
</source_code>
```

Walk through your reasoning first, then provide the structured summary.

Prompt for descriptive summary:

System Prompt

You are a helpful data scientist for software engineering. Your job is to create a descriptive summary and a title explaining a software module's overall behaviour, architecture, and how the components interact within the cluster.

When given a task, always reason through the problem step by step before producing your final answer. This ensures your summary accurately reflects the architecture and interactions rather than surface-level details.

User Prompt

Please create a high-level descriptive summary for the following:

```
<semantic_summaries>
{content of available semantic summaries}
</semantic_summaries>

<descriptive_summaries>
{content of available descriptive summaries}
</descriptive_summaries>
```

Before writing the final output, reason through the following steps:

```

<reasoning>
1. SEMANTIC ANALYSIS: What is the core functionality and logic of this module
   based on the semantic summaries? What are its inputs, outputs, and key
   dependencies?
2. SUBDIRECTORY ANALYSIS: What responsibilities do the subdirectories handle
   based on the descriptive summaries? How do they relate to the top-level
   module?
3. ARCHITECTURE: How is the module structured? What are the layers, and what
   does each own?
4. COMPONENT INTERACTION: How does data flow between the top-level module and
   its subdirectories? Is there shared state? What are the coupling
   characteristics?
5. SUMMARY SYNTHESIS: Given the above, what is the most accurate title and
   concise narrative that captures the module's overall behaviour and design?
</reasoning>

Once you have reasoned through each step, produce the final output in this
format:
**Title:** <title>
**Summary:** <summary>

```

B Parts under focus

- lucene/codecs/src/java/org/apache/lucene/codecs/simpletext
- lucene/codecs/src/java/org/apache/lucene/codecs/blockterms
- lucene/codecs/src/java/org/apache/lucene/codecs/bloom
- lucene/codecs/src/java/org/apache/lucene/codecs/memory
- lucene/codecs/src/java/org/apache/lucene/codecs/blocktreeords